Analyse du marché des NFT Solana

Projet final — Architectures BD et collecte de données · ESEO 2026-2027 · Yann Monkam Nono · dépôt : *[lien Git à compléter]*

1. Problème, utilisateurs, KPI et sources

Utilisateur cible : collectionneur·se ou analyste d'un petit fonds Web3 qui suit quelques collections NFT Solana. **Problème :** le « floor price » affiché par les plateformes est instantané et peut être influencé par quelques acteurs ; il ne dit pas si l'activité vient d'une vraie demande ou de quelques robots ou gros acheteurs. **Décision :** acheter, attendre ou vendre ; se méfier d'un marché porté par peu d'acteurs.

Question centrale : *pour quatre collections NFT Solana de référence sur Magic Eden, comment évoluent les prix et le volume des ventes au fil des collectes quotidiennes, et la demande (ventes et offres d'achat) est-elle portée par une large base d'acheteurs ou par quelques acteurs, souvent automatisés ?*

Cas d'usage et indicateurs

En tant qu'analyste, je veux...	KPI
suivre le prix afin de repérer une tendance	prix médian des ventes, tendance (2de moitié vs 1re moitié de la période), offre médiane, ratio offre / vente
mesurer l'activité afin de juger la liquidité	nombre de ventes, volume en SOL et en USD, nombre d'offres
savoir qui porte la demande afin d'évaluer le risque	part des offres du 1er enchérisseur, part des offres automatisées (pools AMM), part du volume des 10 % d'acheteurs les plus dépensiers, taux d'acheteurs récurrents

Périmètre : ventes (`buyNow`) et offres (`bid`) de mad_lads, okay_bears, claynosaurz et famous_fox_federation. Exclus : annonces, retraits, mises à jour de pools, autres blockchains, métadonnées des NFT. Fréquence utile : quotidienne.

Sources

Source	Rôle	Accès et conditions
API Magic Eden v2 `/collections/{symbol}/activities`	Principale : une ligne par activité (signature, type, prix, date, acheteur, vendeur)	Publique, sans clé, 120 req/min ; Terms of Service, attribution
API CoinGecko `/coins/solana/market_chart`	Enrichissement : cours SOL/USD journalier	Publique ; non bloquante si indisponible

Écartées : OpenSea (clé obligatoire), dataset Kaggle (figé, non reproductible). Pas de scraping. **RGPD :** les wallets (identifiants indirects) sont pseudonymisés par SHA-256 salé dès la zone curated et dans les rejets ; le sel est une variable d'environnement ; le raw (adresses en clair) n'est pas versionné et est conservé 90 jours.

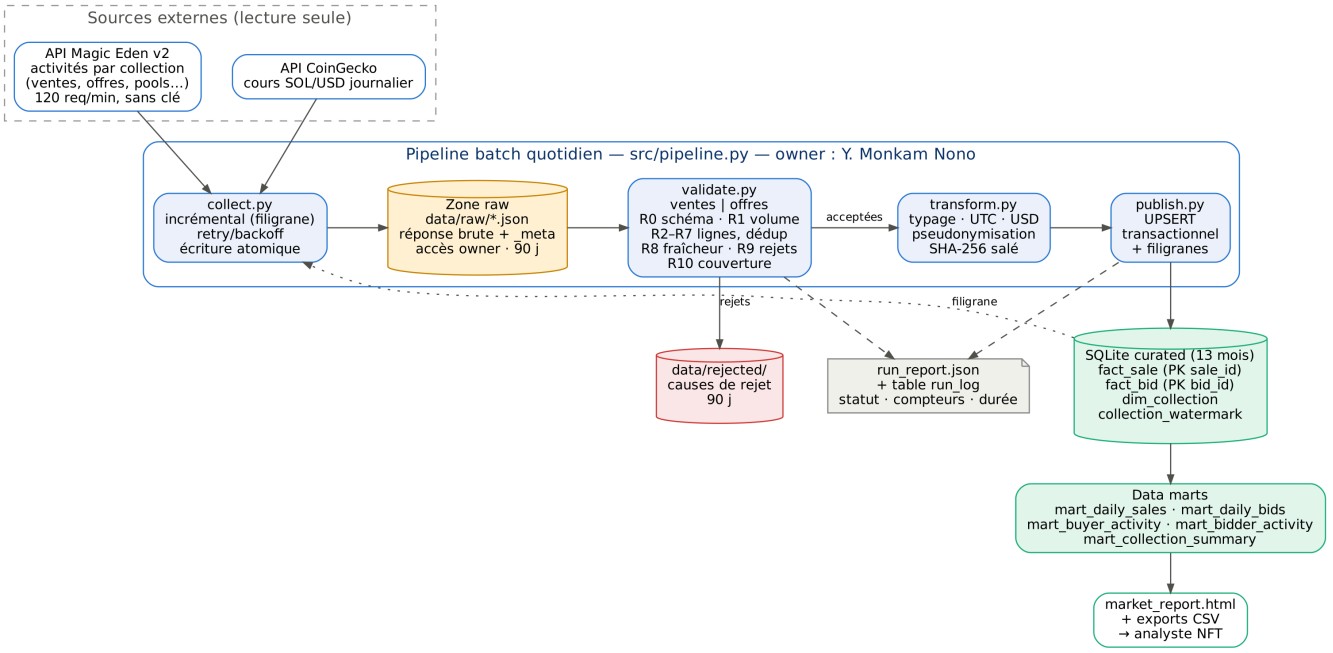
Le profilage du premier run réel a changé le projet

Type d'activité	Nombre	Part
poolUpdate	22 772	38,0 %
bid	20 608	34,3 %
list	16 532	27,6 %
buyNow	52	0,1 %
cancelBid	19	0,0 %
delist	17	0,0 %

Le 07/11/2026, le premier run a collecté **60 000 activités**, mais seulement **52 ventes**. Avec le plafond de 30 pages par collection, un run ne couvre que **12,0 à 41,9 h**.

Décisions : (1) ajouter les **offres d'achat** comme second jeu pour mesurer la demande ; (2) passer à une **collecte incrémentale par filigrane** qui construit l'historique run après run, au lieu de viser 30 jours en une seule fois.

2. Pipeline et architecture



Architecture V2 retenue. La V1 des séances 1-2 (`docs/architecture_v1.png`) : une collection, ventes seules, 5 règles, CSV curated écrasé, sans reprise.

Chaîne de données (DAG batch, `python src/pipeline.py`)

Étape	Rôle	Robustesse
collect.py	Lit le filigrane de chaque collection, pagine du plus récent au plus ancien jusqu'au filigrane (moins 1 h de recouvrement) ; cours SOL/USD. Écrit `data/raw/magiceden_<coll>_<run_id>.json` (réponse brute + `_meta`).	3 essais avec backoff (429, 5xx, réseau) ; écriture atomique ; une collection en échec n'arrête pas les autres
validate.py	Contrôle de schéma, séparation ventes / offres, 11 règles, rejets motivés	blocage si schéma, volume nul ou taux de rejet > 20 %
transform.py	Typage, UTC, conversion USD, pseudonymisation, canal de l'offre (pool AMM / manuelle)	cours manquant → `price_usd` vide + alerte
publish.py	UPSERT des ventes, des offres et des filigranes dans une transaction ; vues mart ; exports CSV ; rapport HTML	tout ou rien ; le filigrane n'avance qu'après succès

Modèle de données (SQLite)

Table	Grain	Clé métier et contraintes
`fact_sale`	une vente d'un NFT	PK `sale_id` = signature ; CHECK prix > 0, acheteur ≠ vendeur ; FK collection
`fact_bid`	une offre sur un NFT ou sur la collection	PK `bid_id` = signature + mint (une transaction peut poser plusieurs offres) ; CHECK canal et portée
`collection_watermark` , `run_log` , `dim_collection`	technique	filigrane incrémental, observabilité, référentiel

Chaque ligne porte `run_id` , `source_file` et `contract_version` : on remonte à la réponse API exacte (lineage). **Couches :** raw (rôle de lake : rejouer) → SQLite curated (rôle de warehouse léger) → vues et CSV mart (usage). Pas de Data Lake objet ni de Warehouse séparé : une seule source, moins de 10 M lignes par an. **Idempotence :** déduplication par clé dans le lot, PK + `ON CONFLICT` en base, exports régénérés depuis la base.

3. Qualité, incidents et observabilité

Activités brutes	Ventes acceptées	Offres acceptées	Rejets	Doublons retirés
60 000	**52**	**20 603**	**0**	**5**

Run 20261107T210929Z sur le lot réel du 07/11/2026 — statut **PASS_WITH_WARNINGS** en 202,0 s de traitement (collecte comprise, limitée par le débit de l'API). 39 340 activités hors périmètre comptées par type et conservées dans le raw. **Collecte incrémentale** suivante (run 20261107T211854Z) : 2 500 activités en 7,4 s au lieu de 60 000 en 202 s, arrêt au filigrane pour les 4 collections, 42 nouvelles offres et 576 lignes du recouvrement d'1 h ignorées car déjà chargées. **Rejeu** du dernier lot (run 20261107T211915Z, `--replay`) : **0 ligne insérée**, 618 déjà présentes.

Règle	Dimension	Condition	Action	Ventes	Offres
R0_SCHEMA	validité	champs du contrat présents	bloquer le run	0 (PASS)	
R1_VOLUME	complétude	au moins une vente ou offre	bloquer le run	0 (PASS)	
R2_REQUIRED	complétude	champs obligatoires non vides (liste par jeu)	rejeter	0 (PASS)	0 (PASS)
R3_TIMESTAMP	validité	blockTime entier entre 2021-03-01 et maintenant+10min	rejeter	0 (PASS)	0 (PASS)
R4_PRICE_RANGE	validité	0 < price <= 100000 SOL	rejeter	0 (PASS)	0 (PASS)
R5_SELF_TRADE	cohérence	buyer != seller	rejeter (auto-vente)	0 (PASS)	–
R6_COLLECTION	cohérence	collectionSymbol = collection interrogée	rejeter	0 (PASS)	0 (PASS)
R7_UNIQUE	unicité	clé métier signature unique dans le lot	dédupliquer et compter	0 (PASS)	5 (WARN)
R8_FRESHNESS	fraîcheur	observation la plus récente < 72 h	alerte	0 (PASS)	0 (PASS)
R9_REJECT_RATE	fiabilité	taux de rejet <= 20%	bloquer la publication	0 (PASS)	0 (PASS)
R10_COVERAGE	complétude	fenêtre collectée jointive avec le run précédent	alerte (trou de collecte daté)	0 (PASS)	

Sur données réelles, R7 a retiré **5 offre(s) renvoyée(s) en double par l'API**. Aucune ligne n'a été rejetée. Les règles R2 à R6 sont démontrées par les tests (anomalies injectées) : chaque rejet garde **toutes** ses causes, son jeu et son fichier source.

Tests d'incident

Incident simulé	Statut	Publié
source indisponible	FAILED_SOURCE_UNAVAILABLE	non
changement schema	FAILED_SCHEMA_CHANGE	non
source vide	FAILED_EMPTY_SOURCE	non
doublons massifs	PASS_WITH_WARNINGS	oui
valeurs impossibles	FAILED_QUALITY	non

Couverture du premier run (R10)

Collection	Plus ancienne	Plus récente	Durée	Arrêt
claynosaurz	06/11 08:30	07/11 21:09	36,7 h	max_pages
famous_fox_federation	06/11 03:20	07/11 21:11	41,9 h	max_pages
mad_lads	06/11 22:01	07/11 21:08	23,1 h	max_pages
okay_bears	07/11 09:12	07/11 21:09	12,0 h	max_pages

Reprise : un run en échec ne touche pas la base et laisse un rapport. Le filigrane n'avance qu'après une publication réussie : le run suivant reprend au dernier point connu. `--replay` retraite le dernier lot sans rappeler l'API. Si le plafond de pages est atteint avant le filigrane, R10 date le trou. **27 tests** `pytest` couvrent les règles, les offres, l'idempotence, le filigrane, les incidents et le collecteur.

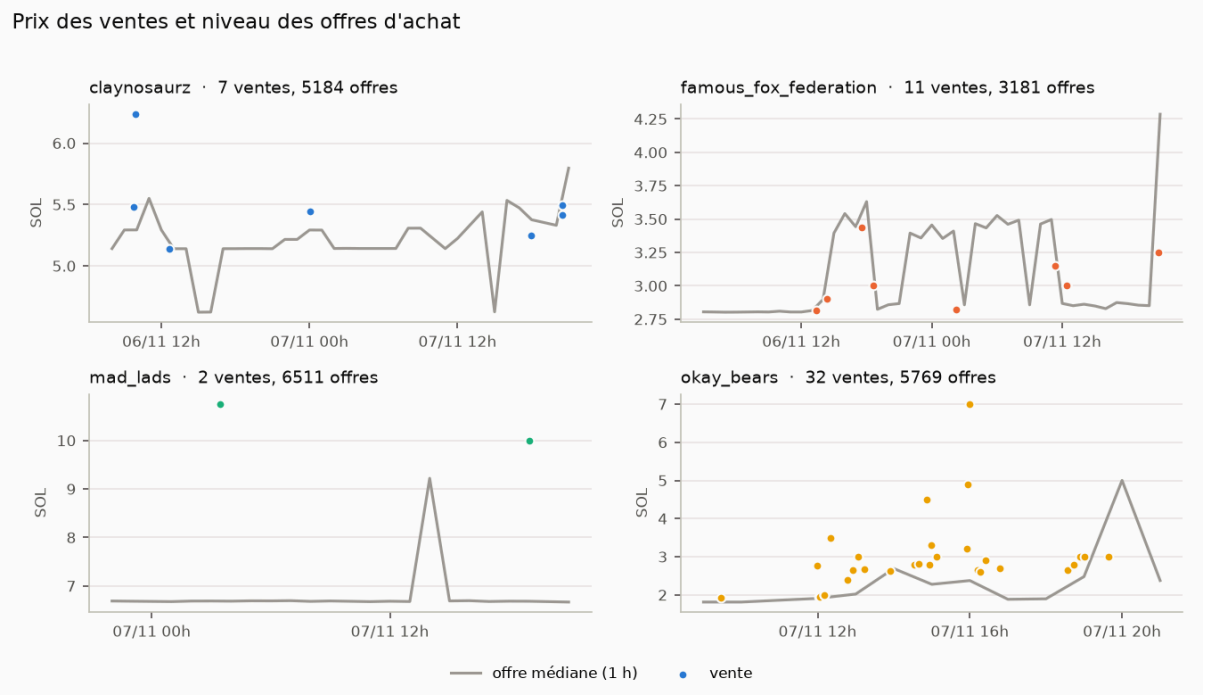
Observabilité : fraîcheur < 72 h (R8), taux de rejet ≤ 2 % OK / ≤ 20 % alerte / > 20 % blocage (R9), couverture (R10), statut et durée de chaque run dans `run_log` et `reports/runs/`. Objectif : les ventes et offres de J-1 publiées avant 8 h, sans trou.

4. Résultat

Données réelles du lot du 07/11/2026 (6 au 7 novembre selon la collection) · 52 ventes, 20 645 offres, ≈ 16 583 USD de volume. Produit consultable : `reports/market_report.html` et `data/marts/*.csv`.

Prix et volume

Collection	Période	Ventes	Volume SOL	Volume USD	Vente médiane	Tendance*	Offre médiane	Offre / vente
claynosaurz	36 h	7	38,5	3 422	5,44	-0,8 %	5,55	102 %
famous_fox_federation	42 h	11	33,4	2 970	3,00	3,4 %	2,86	95 %
mad_lads	23 h	2	20,8	1 842	10,38	–	6,70	64 %
okay_bears	12 h	32	94,0	8 349	2,80	14,6 %	1,88	67 %



Qui porte la demande ?

Collection	Offres	Enchérisseurs	1er enchérisseur	Offres pools AMM	Acheteurs	Récurrents	Top 10 % → volume
claynosaurz	5 184	18	72 %	28 %	7	0 %	16 %
famous_fox_federation	3 181	13	32 %	100 %	7	29 %	36 %
mad_lads	6 511	11	50 %	82 %	2	0 %	52 %
okay_bears	5 769	32	75 %	81 %	20	35 %	27 %

* médiane des ventes de la 2de moitié de la période rapportée à celle de la 1re (calculée seulement avec au moins 3 ventes par moitié).

Lecture

- **Une demande très concentrée et automatisée :** sur okay_bears, un seul enchérisseur pose 75 % des offres ; sur famous_fox_federation, 100 % des offres viennent de pools AMM. Le nombre d'offres mesure donc surtout l'activité de robots, pas une large demande.
- **Peu de ventes :** 52 ventes au total, dont 32 sur okay_bears. Les prix médians restent indicatifs tant que l'historique est court.
- **Écart entre offre et vente :** sur mad_lads, l'offre médiane ne vaut que 64 % de la vente médiane : les enchérisseurs, surtout automatisés, proposent nettement moins que le prix réellement payé.

5. Décisions, limites et perspectives

ADR 001 — architecture retenue

Option	Bénéfice	Coût / risque
Simple : script + CSV écrasé	rapide	pas de rejeu, doublons, pas d'historique ; inadaptée à une source qui ne couvre qu'un jour
Proportionnée (retenue) : batch Python incrémental, raw JSON, SQLite contrainte, marts	rejouable, idempotent, historique cumulatif, sans infrastructure	mono-machine, planification externe
Évolutive : flux temps réel, Kafka, lake S3/Parquet, warehouse, Airflow	latence en secondes, montée en charge	plusieurs services sans exigence qui les justifie

- **Batch incrémental et non streaming :** la décision est quotidienne. Le filigrane donne la continuité d'un flux sans en avoir le coût d'exploitation.
- **SQL (SQLite) et non NoSQL :** les requêtes sont des agrégations relationnelles (par collection × jour, par acheteur) sur un schéma stable. La variabilité de la source (champs absents selon le type) est absorbée par la zone raw JSON et le contrat. Un seul écrivain : pas besoin de serveur PostgreSQL.
- **Ventes + offres dans le même flux :** pas de nouvelle source ni de nouvel appel, mais une mesure de la demande 400 fois plus riche que les ventes seules.
- **Condition de réexamen :** alerte en moins de 5 min, trous répétés détectés par R10, plus de 10 collections, plusieurs utilisateurs distants, ou seconde marketplace.

Gouvernance

Owner : Yann Monkam Nono. Rétention : raw et rejets 90 jours, curated et rapports 13 mois. Contrat versionné (v1.0.0 → v1.2.0 ; la v1.2.0 découle du profilage). Secrets uniquement par variables d'environnement. Accès en écriture réservé au pipeline.

Limites

- L'historique commence au premier run ; un run ne remonte que 11 à 42 h. Sauter un run sur une collection très active peut créer un trou (daté par R10).
- Peu de ventes (≈ 50 par jour sur 4 collections) : les KPI de prix sont fragiles sur une courte période.
- Magic Eden ne couvre qu'une partie du marché Solana (Tensor absent) ; une personne peut avoir plusieurs wallets ; R5 ne voit que l'auto-vente directe.
- Cours SOL/USD journalier ; planification non intégrée (cron ou Planificateur de tâches Windows).

Perspectives

- Planifier un run toutes les 6 h et alerter en cas de statut FAILED.
- Ajouter Tensor avec une couche d'intégration commune ; distinguer les teneurs de marché connus des acheteurs humains.
- Détecter le wash trading par graphe d'échanges entre wallets pseudonymisés ; passer à Parquet partitionné au-delà de quelques millions de lignes.

Usage de l'IA

Claude (Anthropic) a assisté la conception, une première version du code, des tests et de la documentation, et l'analyse du profilage. Le code a été exécuté sur données réelles (Windows, Python 3.13), testé (27 tests, collecte incrémentale, rejeu, 5 incidents) et relu ; l'usage est déclaré dans le README.